

ReadyAt과 Gen — 그림으로 보기

둘 다 시간이 엇갈릴 때 생기는 사고를 막는 숫자 하나씩이다. **ReadyAt**은 리스폰 타이머를, **Gen**은 비동기 스폰 결과를 맡는다. 슬롯마다 돌던 비동기 작업과 CTS를 이 두 숫자로 바꾸면 S1~S3이 생길 자리가 없어진다.

1. ReadyAt — "몇 초부터 다시 스폰해도 되는가"

남은 시간(카운트다운)이 아니라 **알람 시각**을 적어 둔다. 상황: 리스폰 10초, 100초에 회수, 105초에 끄, 112초에 다시 켜

그림 1 · 같은 상황, 옛 방식과 새 방식



보라 구간 = Sector가 스포너를 끈 동안. 옛 방식은 타이머가 "돌고 있는 작업"이라 끝 때 취소해야 하고, 새 방식은 취소할 것이 없다.

한 줄로

ReadyAt은 **알람 시각**이다. 스포너를 꺼도 알람은 지워지지 않고, 다시 켜를 때 이미 지난 시각이면 그 자리에서 올린다.

타입이 하는 일 — 시계를 보고 비교만 한다

그림 2 · now = 108초인 프레임에서, 켜진 스포너의 슬롯 4개



규칙은 하나: `State == Waiting` 이고 `now ≥ ReadyAt` 이면 스폰. 파란 선(now)보다 왼쪽에 있는 알람만 올린다. 예산이 프레임당 1마리라면 슬롯 0은 이번 프레임에, 슬롯 1은 다음 프레임에 나온다. 알람이 이미 지났으니 한 프레임 늦어도 읽는 것이 없다.

2. Gen — "이 스폰 결과가 아직 유효한가"

`MonsterFactory.CreateMonsterByMonsterIDAsync` 는 비동기다. 요청하고 몬스터가 나오기까지 몇 프레임이 걸리고, 그 사이에 스폰너가 꺼졌다 켜질 수 있다

비유 — 카페 번호표

슬롯은 주문할 때마다 번호를 하나 올리고(`Gen++`), 그 번호를 요청에 붙여 보낸다. 주문을 취소할 때도 번호를 올린다. 음료(몬스터)가 나왔을 때 붙여 온 번호가 지금 번호와 다르다면 취소된 주문이므로 바로 버린다. 확인은 정수 비교 한 번이다.



CTS로 취소하면 안 되나?

- Addressable 로드 · Instantiate가 이미 진행된 뒤에는 취소가 깔끔하게 먹지 않을 수 있다
- 결국 "결과가 와 버리면 어떡하지"를 또 처리해야 한다
- 스폰마다 CTS를 만들고 Dispose하는 관리가 다시 생긴다 (옛 S2의 자리)

번호 비교로 하면

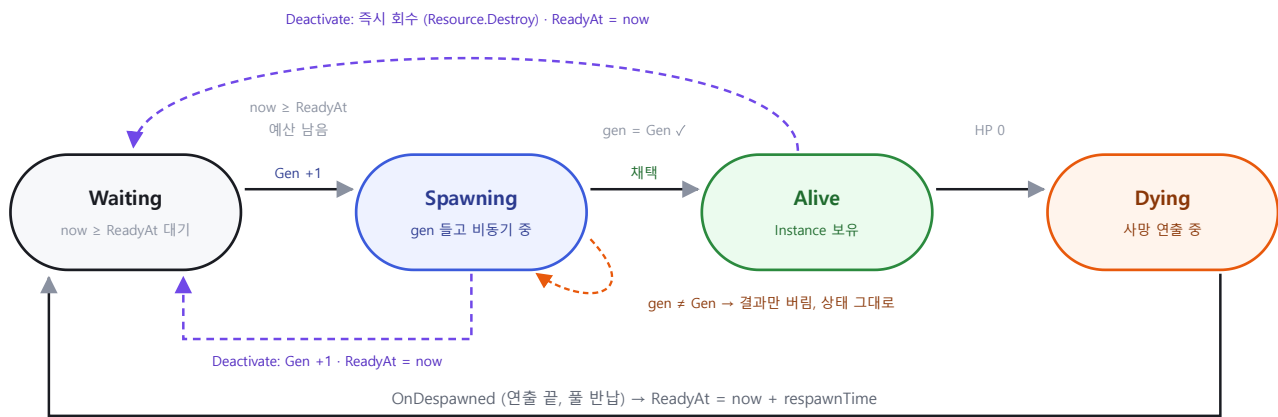
- 결과는 어차피 올 수 있다고 인정하고, 도착한 순간 한 곳에 서만 판정한다
- 확인은 `gen ≠ slot.Gen` 정수 비교 한 번
- 슬롯에 int 하나만 늘어난다. 할당 없음

3. 정리 — 슬롯의 필드 4개와 누가 언제 바꾸는가

상태는 이 네 개가 전부다. 비동기 작업 안에 숨은 상태가 없으니 언제 들여다봐도 슬롯이 무엇을 하는 중인지 보인다

필드	뜻	바꾸는 순간
State	Waiting · Spawning · Alive · Dying	틱(Waiting→Spawning) · 팩토리 반환(→Alive) · 사망(→Dying) · 회수(→Waiting) · Deactivate
ReadyAt	이 시각부터 스폰 가능 (알람 시각)	회수(OnDespawned) 시 <code>now + respawnTime</code> · Deactivate로 되돌릴 때 <code>now</code> · 첫 활성화 (첫 스폰 대기 결정)
Gen	지금 유효한 주문 번호	스폰 요청할 때 <code>+1</code> · Spawning 중 Deactivate로 취소할 때 <code>+1</code>
Instance	슬롯이 관리하는 몬스터	Gen이 맞는 결과가 도착하면 설정 · 회수되면 비움

그림 5 · 상태 기계 위에 ReadyAt과 Gen이 붙는 자리 (실선 = 평소, 점선 = Deactivate)



Waiting에서 Deactivate는 아무것도 안 바꾼다 — ReadyAt이 알람 시각이라 꺼져 있어도 시간은 흐른다.

코드로 보면 (SpawnerManager)

```
void OnUpdate() // 매 프레임 · 켜진 Record만
{
    foreach (slot in 켜진 Record의 슬롯)
        if (slot.State == Waiting && now >= slot.ReadyAt && budget-- > 0)
        {
            slot.Gen++; // 새 주문 번호
            slot.State = Spawning;
            SpawnAsync(slot, slot.Gen).Forget(); // 번호를 들고 출발
        }

    async UniTaskVoid SpawnAsync(SpawnSlot slot, int gen)
    {
        var go = await MonsterFactory.CreateMonsterByMonsterIDAsync(...);
        if (gen != slot.Gen) { Managers.Resource.Destroy(go); return; } // 취소된 주문
        slot.Instance = go; slot.State = Alive;
    }

    void HandleDespawned(BaseMonsterController m) // Dying → Waiting
    {
        slot.Instance = null; slot.State = Waiting;
        slot.ReadyAt = now + slot.RespawnTime; // 알람 맞추기
    }
}
```